

RAPPORT DU PROJET DE CLASSIFICATION D'IMAGES AVEC NEURONES ARTIFICIELLES CRES AVEC NUMPY ET TENSFLOW FLOW

INTRODUCTION

Le Deep Learning est une branche de l'intelligence artificielle basée sur l'utilisation de réseaux de neurones capables d'apprendre à partir de données.

Dans ce projet, nous utilisons la base **Fashion-MNIST**, composée d'images de vêtements de taille **28 × 28 pixels**. L'objectif est de développer un réseau de neurones capable de distinguer les images de la classe **T-shirt/top** des autres catégories.

Le réseau est entièrement implémenté avec **NumPy**, sans utiliser de bibliothèque spécialisée de Deep Learning. Nous mettons en œuvre les principales étapes de l'apprentissage : propagation avant, fonction de coût, rétropropagation et descente de gradient.

Enfin, les performances du modèle sont évaluées à l'aide de la **loss** et de l'**accuracy** sur les données d'entraînement et de test.

1-Présentation de Fashion-MNIST

Fashion-MNIST est une base de données d'images utilisée pour les problèmes de classification. Elle contient des images en niveaux de gris représentant différents types de vêtements.

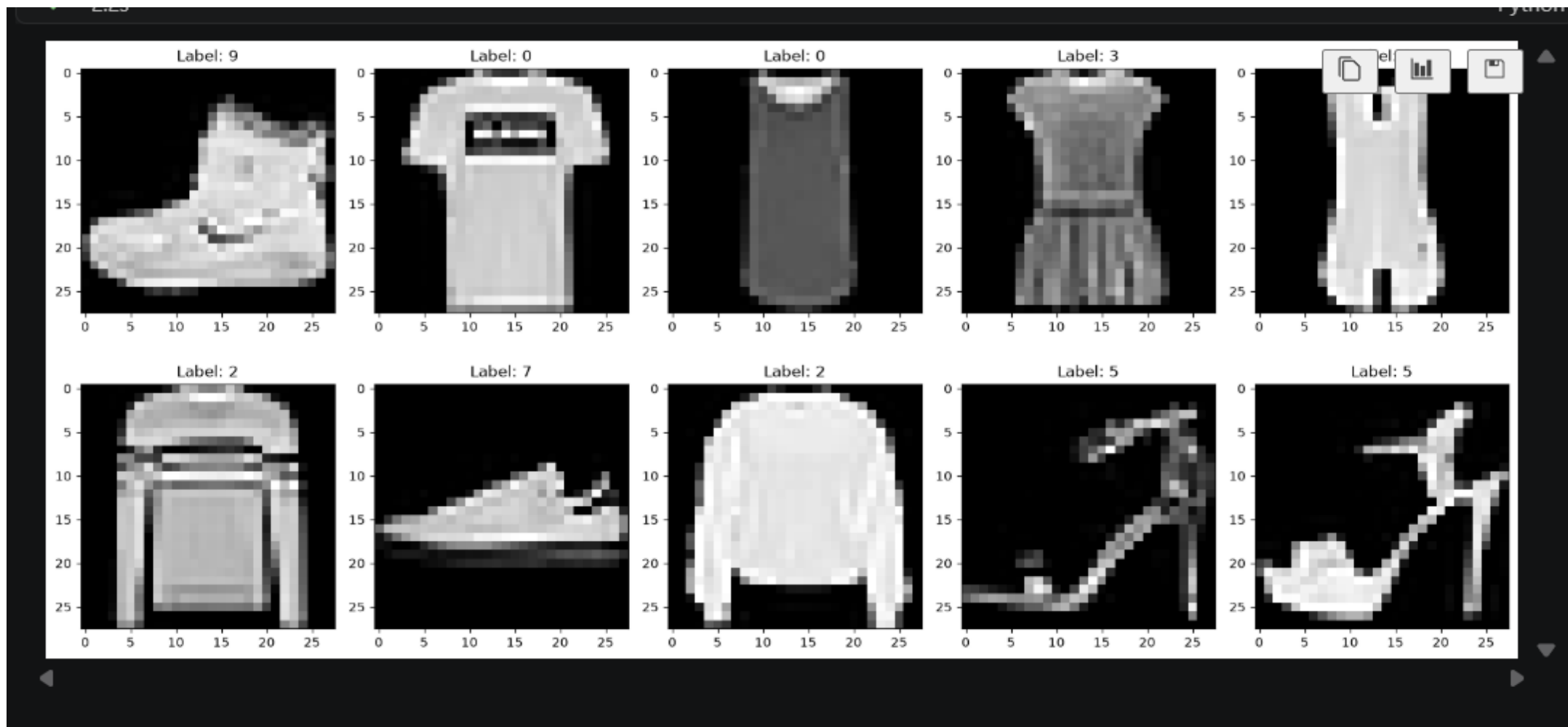
Chaque image possède une résolution de **28 × 28 pixels**, soit **784 pixels**. La base contient **60 000 images d'entraînement** et **10 000 images de test**.

Les images sont réparties à l'origine en **10 classes** :

- T-shirt/top ,Trouser ,Pullover ,Dress ,Coat ,Sandal ,Shirt ,Sneaker ,Bag ,Ankle boot

Dans ce projet, nous simplifions le problème en réalisant une **classification binaire** : la classe **T-shirt/top (label 0)** est considérée comme la classe positive, tandis que toutes les autres classes sont regroupées dans la classe négative.

Figure 1 — Exemples d'images de la base Fashion-MNIST.



2-Chargement et exploration des donnees

Les données Fashion-MNIST ont été téléchargées sous forme de fichiers compressés au format **.gz**. Elles sont ensuite chargées en Python à l'aide du module **gzip** et de la bibliothèque **NumPy**.

Les données obtenues sont réparties en deux ensembles :

- **Entraînement** : 60 000 images et leurs labels.
- **Test** : 10 000 images et leurs labels.

Les dimensions obtenues après le chargement sont :

```
print("X_train :", x_train.shape)
print("y_train :", y_train.shape)
print("X_test  :", x_test.shape)
print("y_test  :", y_test.shape)
```

✓ 0.4s

```
X_train : (60000, 28, 28)
y_train : (60000,)
X_test  : (10000, 28, 28)
y_test  : (10000,)
```

3-Prétraitement des données

4.1 Normalisation des pixels:

Les pixels des images sont initialement représentés par des valeurs comprises entre **0 et 255**. Afin de faciliter l'apprentissage du réseau, ces valeurs sont normalisées entre **0 et 1** en divisant chaque pixel par 255.

```
#Normalisation
X_train = X_train / 255
X_test = X_test / 255
print(X_train.min())
print(X_train.max())
```

7] ✓ 0.2s

0.0
1.0

4.2 Transformation des images

Chaque image possède une dimension de **28 × 28 pixels**. Elle est transformée en un vecteur de **784 valeurs** afin de pouvoir être utilisée comme entrée du réseau de neurones.

Après cette transformation, les données sont transposées afin d'obtenir la forme attendue par notre réseau :

```
print("X_train :", X_train.shape)
print("X_test  :", X_test.shape)
```

18] ✓ 0.0s Python

.. X_train : (784, 60000)
X_test : (784, 10000)

4-Transformation en classification binaire

Fashion-MNIST contient initialement **10 classes**. Dans ce projet, nous avons choisi de simplifier le problème en réalisant une classification binaire.

La classe **0 (T-shirt/top)** est considérée comme la classe positive et reçoit la valeur **1**. Toutes les autres classes reçoivent la valeur **0**.

La transformation est réalisée avec :

```
y_train_binary = (y_train == 0).astype(int)
```

```
y_test_binary = (y_test == 0).astype(int)
```

Les labels sont ensuite remodelés afin d'obtenir une dimension compatible avec les calculs du réseau :

```
... x_train : (784, 60000)
    y_train : (1, 60000)
    x_test  : (784, 10000)
    y_test  : (1, 10000)
```

Cette transformation permet d'utiliser une **fonction sigmoïde en sortie**, dont la valeur permet de déterminer si une image appartient ou non à la classe T-shirt/top.

5-Architecture du reseau de neurone avec numpy

Le réseau de neurones développé dans ce projet est composé d'une couche d'entrée, de trois couches cachées et d'une couche de sortie.

L'architecture utilisée est la suivante :

784 → 32 → 32 → 32 → 1

Couche	Nombre de neurones	Rôle
Entrée	784	Représenter les pixels de l'image
Couche cachée 1	32	Apprendre des caractéristiques de l'image
Couche cachée 2	32	Combiner les caractéristiques apprises
Couche cachée 3	32	Construire une représentation plus complexe
Sortie	1	Prédire la classe binaire

Les **784 neurones d'entrée** correspondent aux 784 pixels de l'image après transformation de la matrice 28*28

Les trois couches cachées utilisent la **fonction d'activation sigmoïde**. La couche de sortie utilise également une sigmoïde afin d'obtenir une valeur comprise entre **0 et 1**, interprétée comme une probabilité d'appartenance à la classe T-shirt/top.

Le réseau contient donc plusieurs paramètres, notamment les matrices de poids W et les biais b , qui sont ajustés progressivement pendant l'apprentissage.

5.1-Initialisation des paramètres

Avant de commencer l'apprentissage, les paramètres du réseau doivent être initialisés. Ces paramètres sont constitués des **poids** W et des **biais** b de chaque couche.

L'initialisation est réalisée à l'aide de `np.random.randn()`. Les dimensions des matrices sont déterminées automatiquement à partir de l'architecture du réseau.

Cette initialisation permet de donner au réseau des valeurs de départ différentes pour les paramètres qui seront ensuite ajustés au cours de l'apprentissage.

La fonction d'initialisation a également été généralisée afin de pouvoir utiliser un nombre variable de couches cachées.

```
def initialisation(dimensions):  
    parametres = {}  
    for c in range(1, len(dimensions)):  
        parametres['w' + str(c)] = np.random.randn(  
            dimensions[c],  
            dimensions[c-1]  
        )  
        parametres['b' + str(c)] = np.random.randn(  
            dimensions[c],  
            1  
        )  
    return parametres
```

5.2-Propagation avant(forward propagation)

La propagation avant consiste à faire passer les données d'entrée à travers les différentes couches du réseau afin d'obtenir une prédiction.

Pour chaque couche, nous calculons d'abord :

$$\begin{aligned} &[\\ &Z^{\{c\}} = W^{\{c\}}A^{\{c-1\}} + b^{\{c\}} \\ &] \end{aligned}$$

Puis nous appliquons la fonction sigmoïde :

$$\begin{aligned} &[\\ &A^{\{c\}} = \frac{1}{1 + e^{-Z^{\{c\}}}} \\ &] \end{aligned}$$

```
def forwar_propagation(X, parametres):  
    activations = {'A0': X}  
    C = len(parametres) // 2  
    for c in range(1, C + 1):  
        z = (  
            parametres['w' + str(c)].dot(  
                activations['A' + str(c-1)]  
            )  
            + parametres['b' + str(c)]  
        )  
        activations['A' + str(c)] = 1 / (1 + np.exp(-z))  
    return activations
```

5.3-Fonction de cout(log loss)

Après la propagation avant, le réseau produit une prédiction. Il faut ensuite mesurer l'écart entre cette prédiction et la valeur réelle.

Pour notre problème de classification binaire, nous utilisons la **Log Loss**, également appelée entropie croisée binaire.

Sa formule est :

$$J = -\frac{1}{m} \sum_{i=1}^m \left[y_i \log(A_i) + (1-y_i) \log(1-A_i) \right]$$

où :

- m représente le nombre d'exemples ;
- y représente la vraie valeur ;
- A représente la prédiction du réseau.

```
#Fonction cout
def log_loss(y, activations):

    m = y.shape[1]

    A = activations['A' + str(len(activations) - 1)]

    epsilon = 1e-15

    A = np.clip(A, epsilon, 1 - epsilon)

    loss = -(1/m) * np.sum(
        y * np.log(A) +
        (1-y) * np.log(1-A)
    )

    return loss
```

Plus la valeur de la **Loss** est faible, plus les prédictions du réseau sont proches des valeurs réelles.

Dans notre implémentation, nous utilisons également `np.clip()` afin d'éviter les problèmes numériques lorsque la prédiction est très proche de **0 ou 1**.

5.4-Retropopagation(back propagation)

La rétropropagation est utilisée pour calculer les **gradients** des poids et des biais du réseau. Elle permet de déterminer dans quelle direction les paramètres doivent être modifiés afin de réduire la fonction de coût.

Le calcul commence par la couche de sortie. Pour notre problème de classification binaire : $dZ^{\{L\}} = A^{\{L\}} - y$

Les gradients des poids et des biais sont ensuite calculés avec : $dW^{\{c\}} = 1/m * dZ^{\{c\}} (A^{\{c-1\}})^T$

$db^{\{c\}} = 1/m * \sum dZ^{\{c\}}$

Le calcul est ensuite effectué **de la dernière couche vers la première**. Pour les couches cachées, le gradient est propagé à travers la fonction sigmoïde.

Cette étape permet donc de déterminer l'influence de chaque paramètre sur l'erreur finale du réseau.

Les gradients obtenus sont ensuite utilisés lors de l'étape suivante : la **descente de gradient**.

```

def back_propagation(y, parameters, activations):

    gradient = {}

    C = len(parameters) // 2

    m = y.shape[1]

    dz = activations['A' + str(C)] - y

    for i in reversed(range(1, C + 1)):

        gradient['dw' + str(i)] = (
            1/m
        ) * np.dot(
            dz,
            activations['A' + str(i-1)].T
        )

        gradient['db' + str(i)] = (
            1/m
        ) * np.sum(
            dz,
            axis=1,
            keepdims=True
        )

        dz = (
            parameters['w' + str(i)].T.dot(dz)
            * activations['A' + str(i-1)]
            * (1 - activations['A' + str(i-1)])
        )

    return gradient

```

5.5- Mise à jour des paramètres par descente de gradient

- Après la rétropropagation, nous obtenons les gradients des poids et des biais. Ces gradients indiquent **dans quelle direction les paramètres doivent être modifiés pour réduire l'erreur**.

La mise à jour s'effectue avec les formules : $W(c) \leftarrow W(c) - \eta dw(c)$; $b(c) \leftarrow b(c) - \eta db(c)$

où :

- W représente les poids ;
- B représente les biais ;
- dw et db sont les gradients calculés par rétropropagation ;
- η représente le **taux d'apprentissage** (*learning rate*).
- Dans notre projet, le taux d'apprentissage utilisé est **0,1**.

À chaque itération, les paramètres sont donc corrigés à partir des gradients. En répétant ce processus, le réseau cherche progressivement à **minimiser la fonction de coût**.

```
#mise a jour des parametre
def update(parametres, gradient, learning_rate):

    C = len(parametres) // 2

    for c in range(1, C + 1):

        parametres['w' + str(c)] = (
            parametres['w' + str(c)]
            - learning_rate * gradient['dw' + str(c)]
        )

        parametres['b' + str(c)] = (
            parametres['b' + str(c)]
            - learning_rate * gradient['db' + str(c)]
        )

    return parametres
```

5.6. Entraînement du réseau de neurones

L'entraînement du réseau consiste à répéter plusieurs fois les différentes étapes de l'apprentissage afin d'ajuster progressivement les poids et les biais.

À chaque itération, les opérations suivantes sont réalisées :

- **Propagation avant** : calcul des prédictions du réseau.
- **Calcul de la Loss** : mesure de l'erreur entre les prédictions et les valeurs réelles.
- **Rétropropagation** : calcul des gradients des paramètres.
- **Mise à jour des paramètres** : modification des poids et des biais avec la descente de gradient.

Pour suivre l'évolution de l'apprentissage, nous enregistrons à chaque itération :

- la **Loss d'entraînement** ;
- la **Loss de test** ;
- l'**Accuracy d'entraînement** ;
- l'**Accuracy de test**.

Pour notre expérimentation, nous avons utilisé un sous-ensemble de **1 000 images d'entraînement** et **50 itérations**, afin de réduire le temps de calcul lors des tests du réseau.

L'architecture utilisée est : 784 → 32 → 32 → 32 → 1

Le processus d'entraînement permet ainsi au réseau d'ajuster progressivement ses paramètres afin d'améliorer ses prédictions.

```
Iteration 0 | Loss train : 0.6359 | Loss test : 0.6363 | Accuracy train : 0.6620 | Accuracy test : 0.6576
```

5.7-Evaluation et Model

Après l'entraînement, les performances du réseau ont été évaluées sur les données d'entraînement et sur les données de test.

Pour cette expérimentation, le réseau a été entraîné sur **1 000 images** pendant **50 itérations**. Les résultats obtenus sont les suivants :

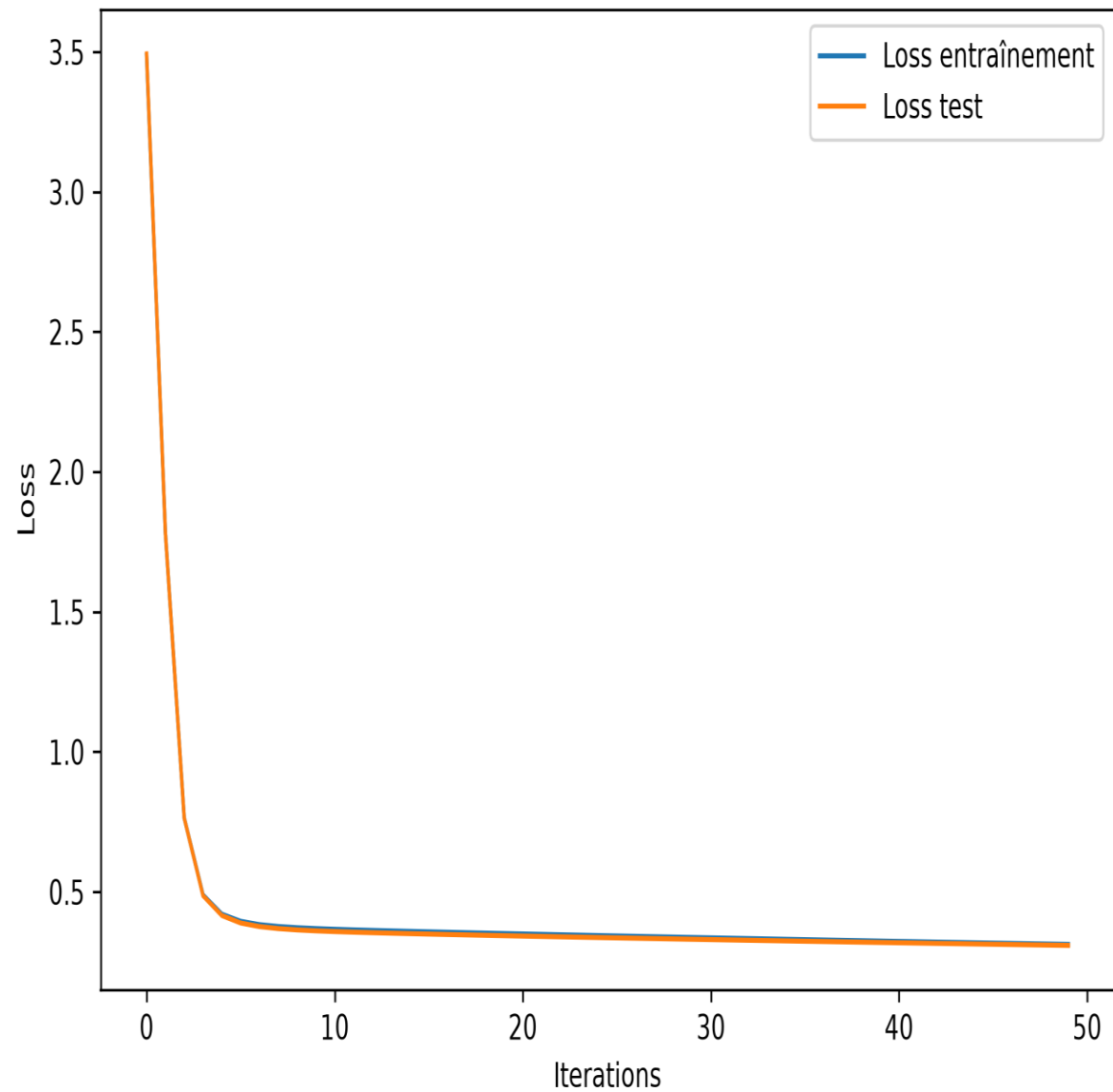
Mesure	Résultat
Accuracy entraînement	88,6 %
Accuracy test	89,0 %

Les résultats montrent que le réseau obtient une accuracy proche de **89 %** sur les données utilisées pour l'évaluation.

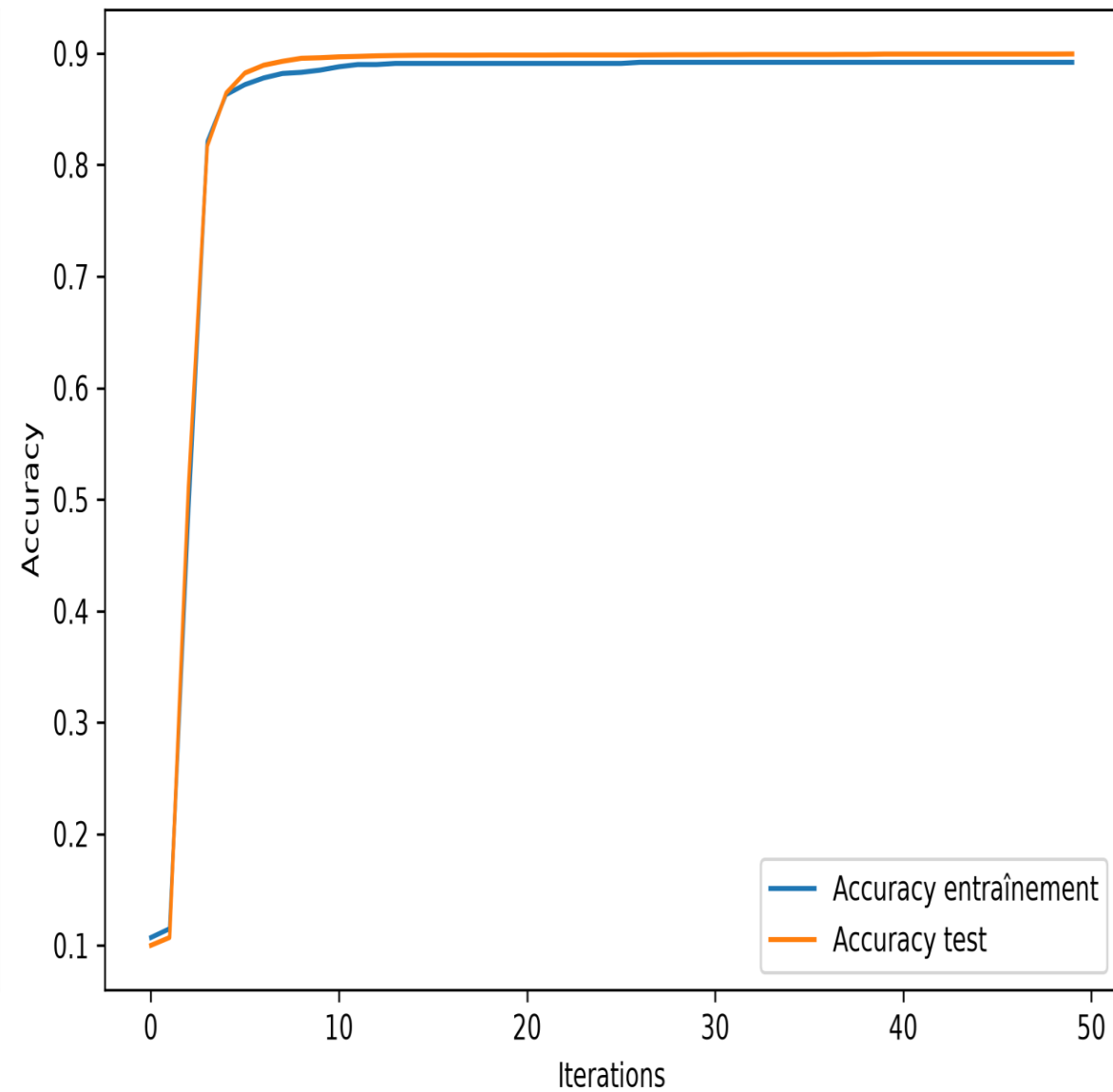
L'accuracy de test est légèrement supérieure à celle d'entraînement dans cette expérimentation. Cette différence reste faible et peut être liée au nombre limité d'itérations et à l'échantillon utilisé pour l'entraînement.

Les courbes de **Loss** et d'**Accuracy** permettent également de suivre l'évolution des performances du réseau au cours des 50 itérations.

Évolution du Loss



Évolution de l'Accuracy



5.8-Analyse des courbes

A-Courbe de gauche : Évolution du Loss (Fonction de perte)

- **Comportement** : Le *Loss* d'entraînement (bleu) et le *Loss* de test (orange) chutent de manière spectaculaire dès les 5 premières itérations, passant de **3.5 à moins de 0.5**.
- **Stabilisation** : Après l'itération 5, la descente ralentit pour se stabiliser de façon asymptotique autour d'une valeur basse d'environ **0.3** jusqu'à l'itération 50.
- **Interprétation** : Le modèle apprend très vite et ajuste ses poids de manière optimale. Le fait que la courbe de test suive parfaitement la courbe d'entraînement prouve que le modèle généralise d'une excellente façon sur de nouvelles données.

B-Courbe de droite : Évolution de l'Accuracy (Précision)

- **Comportement** : En parfaite corrélation avec le Loss, l'exactitude s'envole instantanément. Elle commence à la valeur d'un choix aléatoire (environ **10%** ou **0.1**) et atteint plus de **85%** (0.85) en à peine 5 itérations.
- **Convergence** : Le modèle atteint un excellent plateau de performance stable à **90% d'accuracy (0.9)**, conservé jusqu'à la fin de l'entraînement.
- **Interprétation** : Les performances sur les données de test sont identiques (voire infinitésimalement supérieures au départ) à celles d'entraînement. C'est le comportement recherché en Deep Learning : pas d'écart (*gap*) entre entraînement et validation, donc le modèle n'a pas fait de "par cœur".

5.9. Limites et perspectives

15.1 Limites

Ce projet présente certaines limites.

Tout d'abord, l'expérimentation finale a été réalisée sur un sous-ensemble de **1 000 images d'entraînement** au lieu des 60 000 disponibles. De plus, le réseau a été entraîné pendant seulement **50 itérations**, principalement afin de réduire le temps de calcul.

Ensuite, le problème a été simplifié en une **classification binaire**. Le modèle ne distingue donc pas les 10 catégories originales de Fashion-MNIST.

Enfin, toutes les couches utilisent actuellement la fonction d'activation **sigmoïde**, ce qui peut limiter les performances d'un réseau plus profond.

15.2 Perspectives

Plusieurs améliorations pourraient être envisagées :

- entraîner le réseau sur l'ensemble des données d'entraînement ;
- augmenter le nombre d'itérations ;
- tester différentes architectures et différents nombres de neurones ;
- utiliser d'autres fonctions d'activation, comme **ReLU** ;
- réaliser une classification des **10 classes originales** de Fashion-MNIST ;
- comparer les résultats avec des modèles utilisant des bibliothèques spécialisées de Deep Learning.

Ces améliorations permettraient d'étudier plus largement les performances du réseau et de mieux comprendre l'influence de ses différents paramètres.

6. Réseau de neurones développé avec TensorFlow/Keras

Dans cette partie, nous avons développé un réseau de neurones artificiels avec **TensorFlow et Keras** afin de réaliser une classification multiclassées des images du jeu de données **Fashion-MNIST**.

Chaque image possède une taille de **28 × 28 pixels**, soit **784 valeurs** après aplatissement. Le réseau est composé d'une couche d'entrée, de deux couches cachées et d'une couche de sortie.

L'architecture du réseau est donc :

84 → 128 → 64 → 10

La couche **Flatten** transforme l'image de dimension 28 × 28 en un vecteur de **784 valeurs**. Les deux couches cachées utilisent la fonction d'activation **ReLU**, qui permet au réseau d'apprendre des relations non linéaires. Enfin, la couche de sortie contient **10 neurones**, correspondant aux dix catégories de Fashion-MNIST, et utilise **Softmax** afin de produire une probabilité pour chaque classe.

Le modèle a été compilé avec l'optimiseur **Adam** et la fonction de perte **Sparse Categorical Crossentropy**, adaptée à une classification multiclassés avec des labels entiers.

L'entraînement a été réalisé pendant **10 epochs**, avec une taille de lot (batch_size) de **32** et **10 % des données d'entraînement réservées à la validation**.

À l'issue de l'entraînement, le modèle a obtenu une **accuracy de 91,01 % sur les données d'entraînement, 88,48 % sur les données de validation** et **88,23 % sur les données de test**.

6-Comparaison entre l'implémentation NumPy et TensorFlow

Dans ce projet, deux implémentations d'un réseau de neurones ont été réalisées sur le jeu de données Fashion-MNIST : une première implémentation avec **NumPy** et une seconde avec **TensorFlow/Keras**. L'objectif est de montrer la différence entre une implémentation manuelle et l'utilisation d'un framework de Deep Learning.

A. Préparation des données:

Avec NumPy, les images ont été normalisées puis transformées manuellement afin d'être compatibles avec les calculs matriciels du réseau.

```
X_train = X_train / 255
```

```
X_test = X_test / 255
```

```
X_train = X_train.reshape(X_train.shape[0], -1).T
```

```
X_test = X_test.reshape(X_test.shape[0], -1).T
```

Dans cette implémentation, les images sont aplaties et transposées afin d'obtenir une matrice de dimension $784 \times m$, où m représente le nombre d'exemples.

Avec TensorFlow, la normalisation est également réalisée avec NumPy :

```
X_train = X_train.astype("float32") / 255.0
```

```
X_test = X_test.astype("float32") / 255.0
```

B. Création du réseau

Dans l'implémentation NumPy, les poids et les biais sont créés manuellement. Il faut ensuite écrire les calculs de propagation avant, de rétropropagation et de mise à jour des paramètres.

Dans TensorFlow/Keras, la création du réseau est beaucoup plus concise :

```
model = tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=(28, 28)), tf.keras.layers.Dense(128,
activation="relu"), tf.keras.layers.Dense(64, activation="relu"), tf.keras.layers.Dense(10, activation="softmax")
])
```

La classe `Sequential` permet d'empiler les différentes couches du réseau.

La couche : transforme chaque image 28×28 en un vecteur de 784 valeurs.

Les couches :

```
Dense(128, activation="relu")
```

```
Dense(64, activation="relu")
```

créent les deux couches cachées du réseau. Contrairement à NumPy, nous n'avons pas besoin de créer directement les matrices de poids et les vecteurs de biais : **Keras les initialise automatiquement.**

Enfin :

```
Dense(10, activation="softmax")
```

constitue la couche de sortie. Elle possède 10 neurones correspondant aux 10 catégories de Fashion-MNIST.

C. Propagation avant

Avec **NumPy**, la propagation avant est calculée manuellement à l'aide des multiplications matricielles, des biais et des fonctions d'activation.

Avec **TensorFlow/Keras**, ces calculs sont automatiquement effectués par les couches Dense. Il suffit de définir l'architecture du réseau.

D. Apprentissage

Avec NumPy, la fonction de coût, la rétropropagation et la descente de gradient ont été programmées manuellement.

Avec TensorFlow/Keras, ces opérations sont automatisées lors de la compilation et de l'entraînement :

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Puis :

```
history = model.fit( X_train, y_train, epochs=10, batch_size=32, validation_split=0.1 )
```

TensorFlow gère automatiquement la propagation, le calcul des gradients et la mise à jour des poids

E. Évaluation

L'évaluation est également simplifiée avec :

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

Le modèle a obtenu une **accuracy de 88,23 % sur les données de test**.

F. Comparaison

L'implémentation **NumPy** permet de comprendre en détail le fonctionnement interne d'un réseau de neurones, car les calculs sont réalisés manuellement.

Avec **TensorFlow/Keras**, ces calculs sont automatisés, ce qui permet de développer plus facilement des modèles de Deep Learning plus complexes.

C onclusion

Ce projet nous a permis de mettre en œuvre et de comparer deux approches pour la classification d'images de la base **Fashion-MNIST** : une implémentation manuelle avec **NumPy** et une implémentation avec **TensorFlow/Keras**.

Avec **NumPy**, nous avons réalisé manuellement les principales étapes d'un réseau de neurones : le prétraitement des données, l'initialisation des paramètres, la propagation avant, le calcul de la fonction de coût, la rétropropagation et la descente de gradient. Le modèle a obtenu une **accuracy de 88,6 % sur l'entraînement et 89,0 % sur le test**.

Avec **TensorFlow/Keras**, nous avons utilisé des outils permettant d'automatiser ces différentes étapes et de construire un réseau plus facilement. Le modèle a obtenu une **accuracy de 91,01 % sur l'entraînement, 88,48 % sur la validation et 88,23 % sur le test**.

Cette comparaison nous a permis de mieux comprendre le fonctionnement interne des réseaux de neurones grâce à NumPy, tout en découvrant l'intérêt de TensorFlow/Keras pour développer plus rapidement des modèles de **Deep Learning**. Ce projet constitue ainsi une première expérience pratique dans la conception, l'entraînement et l'évaluation de réseaux de neurones pour la classification